TECHNICAL REPORT

YouView — User Management Platform

November 2025 · Full-Stack Web App · Node.js · Express · MySQL · EJS

Anshumaan Sai Patnaik

Overview

YouView is a full-stack user-management application backed by a relational MySQL database. Visitors can browse a directory of users, view an individual profile, sign up for an account, sign in to see their own details, and edit or delete their account. The interface is rendered server-side with EJS and enhanced with a small amount of client-side JavaScript for the sidebar, modals, and live character counters.

Where the earlier projects used in-memory data, YouView is built around a real database with a normalised, two-table schema. The emphasis was on writing correct, safe SQL — joins, parameterized queries, uniqueness constraints, and multi-table operations — behind a clean set of RESTful routes.

Technology Stack

Node.js + Express	Application server, routing, and request handling.
MySQL (mysql2)	Relational data store accessed through the *mysql2* driver with parameterized queries.
EJS	Server-side templates for the directory, profile, sign-up, and account views.
method-override	Enables *PATCH* and *DELETE* requests from standard HTML forms.
uuid	Generates a stable unique identifier for each user.
@faker-js/faker	Seeds the database with several hundred realistic sample users for development.
HTML / CSS / JS	Markup, custom styling, and client-side interactivity (sidebar, modals, counters).

Database Design

Data is split across two related tables joined on a shared **userID**:

- **users** — core account fields: *userID*, *username*, *emailID*, and *password*.
- **get_user** — profile detail: the user's *about* text, linked back to *users* by *userID*.

This one-to-one split keeps the core credentials separate from the descriptive profile data, and every read that needs the full profile combines the two tables with a JOIN. The database was populated during development by generating several hundred sample users with faker and bulk-inserting them.

Architecture & Key Logic

Express handles routing with the same middleware foundation as before — form-body parsing, method-override for the full set of HTTP verbs, static assets, and EJS as the view engine. Every database call uses

parameterized queries (`?` placeholders), so user input is never concatenated into SQL — closing off SQL-injection and keeping the queries readable.

Authentication via a joined query

Signing in matches a username and password against the **users** table while pulling the matching profile from **get_user** in a single joined query. An empty result set means the credentials didn't match, so the route responds accordingly instead of rendering a profile.

```
// Sign-in: join both tables and match credentials (parameterized)
let q = `SELECT users.userID, users.username, users.emailID, get_user.about
         FROM users
         JOIN get_user ON users.userID = get_user.userID
         WHERE users.username = ? AND users.password = ?`;

connection.query(q, [username, password], (err, result) => {
  if (err) throw err;
  if (result.length === 0) return res.send("You're not found. Try again!");
  res.render("you.ejs", { yourData: result[0] });
});
```

Enforcing uniqueness

Usernames and email addresses must stay unique. Before any insert or update, the app first checks whether the proposed value already exists and returns an HTTP **409 Conflict** with a clear message if it does. The update route is careful to only run that check for the fields that actually changed — if a user edits their *about* text but keeps the same username and email, no needless uniqueness lookups are performed.

Multi-table operations

Because account data lives in two tables, creating and deleting a user touches both. Sign-up is a two-step flow: the first request validates the username and collects the remaining details, and the second inserts into **users** and then **get_user**. Deletion runs in the reverse order — the dependent **get_user** row is removed first, then the **users** row — so the relationship between the tables is never left dangling.

```
// Delete across both tables, dependent row first
let q1 = `DELETE FROM get_user WHERE userID = ?`;
connection.query(q1, [id], (err) => {
  if (err) throw err;
  let q2 = `DELETE FROM users WHERE userID = ?`;
  connection.query(q2, [id], (err) => {
    if (err) throw err;
    res.redirect("/home");
  });
});
```

Interface

The front end pairs the EJS views with vanilla JavaScript for interactivity: a slide-out sidebar with sign-in / sign-up modals on the directory page, an in-place edit popup on the account page, and live character counters on the bound text fields. After any create, update, or delete the server redirects back to the directory, so the user always returns to a fresh, consistent view of the data.